

CS 113

Week 4 Lecture 1



Today's Topics

Homework #2

Stack Diagrams

Recursion

Stack Diagrams for Recursive Functions

Homework #2

Homework #2: due Thurs 9/24 at 11:59pm.

Exercise: Draw stack diagram

```
public class Negative {  
  
    public static double negative(double x) {  
        double value = x * -1;  
        return value;  
    }  
  
    public static void main(String[] args) {  
        double negativeValue = 0;  
        negativeValue = negative(-3.4);  
    }  
}
```

Exercise: Draw stack diagram

```
public class Abs {  
  
    public static double abs(double x) {  
        if (x < 0) {  
            return -x;  
        }  
        return x;  
    }  
  
    public static void main(String[] args) {  
        double absValue = 0;  
        absValue = abs(5.4);  
    }  
}
```

Lab Methods (& Stack Diagrams)

`getUserInput()`

`sumTwo()`

`sumThree()`

What is different here?

```
// Function: area
// Description: computes the area of a rectangle
// Input: width (double)
// Input: height (double)
// returns (double), the area as width * height
// side effects: none
public static double area(double width, double height) {
    return width * height;
}
```

```
// Function: area
// Description: computes the area of a rectangle
// Input: width (double)
// Input: height (double)
// returns (none)
// Side effect: prints the area to the console
public static void area(double width, double height)
{
    double a = width * height;
    System.out.println("Area is " + a);
}
```

Warning: don't confuse printing with returning

```
// Function: area
// Description: computes the area of a rectangle
// Input: width (double)
// Input: height (double)
// returns (double), the area as width * height
// side effects: none
public static double area(double width, double height) {
    return width * height;
}
```

```
// Function: area
// Description: computes the area of a rectangle
// Input: width (double)
// Input: height (double)
// returns (none)
// Side effect: prints the area to the console
public static void area(double width, double height)
{
    double a = width * height;
    System.out.println("Area is " + a);
}
```

Recursion

Recursion: When a method calls itself to solve a smaller version of the same problem.

A recursive method needs:

1. **Base case:** when do we stop?
2. **Recursive case:** how do we make the problem smaller?

Example: Countdown

Print the numbers from 3 to 1 and then print “blastoff!”

Print the numbers from 10 to 1 and then print “blastoff!”

Print the numbers from 100 to 1 and then print “blastoff!”

Print from n to 1 and then print “blastoff!”

Why do we need a base case?

Recursive call returns & the execution stack

Returning a value

Recipe for Recursive Methods

Loops

Exercise

Suppose we wanted to ask the user for 6 numbers (int) and output their sum?

Loops

- Easy way to repeat some computation
- Two kinds of loops:
 - While
 - For
- Loops repeat block of code until the condition becomes false

Example: While Loop

```
int val = 0;
int sum = 0;

int count = 0;
while (count < 6) {
    System.out.print("Enter a number: ");
    val = sc.nextInt();
    sum = sum + val;
    count = count + 1;
}
System.out.println("The sum is "+sum);
```

Tracing Loops

```
int sum = 1;
int count = 0;
while (count < 3) {
    sum = sum + 2;
    count = count + 1;
}
```

Iteration	Count < 6	count	sum

Tracing Loops

```
int sum = 1;
int count = 0;
while (count < 3) {
    sum = sum + 2;
    count = count + 1;
}
```

Iteration	Count < 6	count	sum
0	T	0	1

Tracing Loops

```
int sum = 1;
int count = 0;
while (count < 3) {
    sum = sum + 2;
    count = count + 1;
}
```

Iteration	Count < 6	count	sum
0	T	0	1
1	T	1	3

Tracing Loops

```
int sum = 1;
int count = 0;
while (count < 3) {
    sum = sum + 2;
    count = count + 1;
}
```

Iteration	Count < 6	count	sum
0	T	0	1
1	T	1	3
2	T	2	5

Tracing Loops

```
int sum = 1;  
int count = 0;  
while (count < 3) {  
    sum = sum + 2;  
    count = count + 1;  
}
```

Iteration	Count < 6	count	sum
0	T	0	1
1	T	1	3
2	T	2	5
3	T	3	7

Exercise: Tracing loops

```
int sum = 10;  
int count = 0;  
while (count < 6) {  
    sum = sum - 1;  
    count = count + 2;  
}
```

Iteration	Count < 6	count	sum

Accumulator pattern

Idea: Repeatedly update a variable (typically in a loop)

Pattern:

1. Initialize accumulator variable
2. Loop until done
 1. Update the accumulator variable

Convenience syntax: Assignment

Because updating variable values is so common, language such as Java provide shorthand syntax for it

- Analogy: contractions in English

`sum = sum + 2`

`count = count + 1`

`count = count - 1`

`product = product * 2`

`divisor = divisor / 2`

`message = message + "lol!"`

Convenience syntax: Assignment

Because updating variable values is so common, language such as Java provide shorthand syntax for it

- Analogy: contractions in English

sum = sum + 2	
count = count + 1	
count = count - 1	
product = product * 2	
divisor = divisor / 2	
message = message + " lol"	

Convenience syntax: Assignment

Because updating variable values is so common, language such as Java provide shorthand syntax for it

- Analogy: contractions in English

sum = sum + 2	sum += 2
count = count + 1	
count = count - 1	
product = product * 2	
divisor = divisor / 2	
message = message + " lol"	

Convenience syntax: Assignment

Because updating variable values is so common, language such as Java provide shorthand syntax for it

- Analogy: contractions in English

<code>sum = sum + 2</code>	<code>sum += 2</code>
<code>count = count + 1</code>	<code>count += 1</code>
<code>count = count - 1</code>	
<code>product = product * 2</code>	
<code>divisor = divisor / 2</code>	
<code>message = message + " lol"</code>	

Convenience syntax: Assignment

Because updating variable values is so common, language such as Java provide shorthand syntax for it

- Analogy: contractions in English

<code>sum = sum + 2</code>	<code>sum += 2</code>
<code>count = count + 1</code>	<code>count += 1</code>
<code>count = count - 1</code>	<code>count -= 1</code>
<code>product = product * 2</code>	
<code>divisor = divisor / 2</code>	
<code>message = message + " lol"</code>	

Convenience syntax: Assignment

Because updating variable values is so common, language such as Java provide shorthand syntax for it

- Analogy: contractions in English

<code>sum = sum + 2</code>	<code>sum += 2</code>
<code>count = count + 1</code>	<code>count += 1</code>
<code>count = count - 1</code>	<code>count -= 1</code>
<code>product = product * 2</code>	<code>product *= 2</code>
<code>divisor = divisor / 2</code>	
<code>message = message + " lol"</code>	

Convenience syntax: Assignment

Because updating variable values is so common, language such as Java provide shorthand syntax for it

- Analogy: contractions in English

<code>sum = sum + 2</code>	<code>sum += 2</code>
<code>count = count + 1</code>	<code>count += 1</code>
<code>count = count - 1</code>	<code>count -= 1</code>
<code>product = product * 2</code>	<code>product *= 2</code>
<code>divisor = divisor / 2</code>	<code>divisor /= 2</code>
<code>message = message + " lol"</code>	

Convenience syntax: Assignment

Because updating variable values is so common, language such as Java provide shorthand syntax for it

- Analogy: contractions in English

<code>sum = sum + 2</code>	<code>sum += 2</code>
<code>count = count + 1</code>	<code>count += 1</code>
<code>count = count - 1</code>	<code>count -= 1</code>
<code>product = product * 2</code>	<code>product *= 2</code>
<code>divisor = divisor / 2</code>	<code>divisor /= 2</code>
<code>message = message + " lol"</code>	<code>message += " lol"</code>

Exercise: Write a program that computes powers of 2

Write a program, LoopPow2.java, that computes powers of twos. For example,

```
$ java LoopPow2
Enter an exponent: 0
2 to the power of 0 is 1
```

```
$ java LoopPow
Enter an exponent: 1
2 to the power of 1 is 2
```

```
$ java LoopPow
Enter an exponent: 4
2 to the power of 4 is 16
```

Example: For Loop

```
int val = 0;
String valStr = "";
int sum = 0;

for (int count = 0; count < 6; count = count +1) {
    System.out.print("Enter a number: ");
    valStr = System.console().readLine();
    val = Integer.parseInt(valStr);
    sum = sum + val;
}
System.out.println("The sum is "+sum);
```

Example: For Loop

initialize

condition

update

```
for (int count = 0; count < 6; count = count + 1) {  
  
}
```

Exercise: Tracing loops

```
String pattern = "";  
for (int i = 0; i < 3; i++) {  
    pattern = pattern + "*";  
}  
System.out.println(pattern);
```

Iteration	i < 3	i	pattern

Exercise: Tracing loops

```
String pattern = "";  
for (int i = 0; i < 3; i++) {  
    pattern = pattern + "*";  
}  
System.out.println(pattern);
```

Iteration	i < 3	i	pattern
0	T	0	""
1	T	1	"*"
2	T	2	"**"
3	F	3	"***"

Exercise: LoopPattern.java

```
$ java LoopPattern
```

```
Enter a length: 5
```

```
*_*_*
```

```
$ java LoopPattern
```

```
Enter a length: 10
```

```
*_*_*_*_*_*
```

```
$ java LoopPattern
```

```
Enter a length: 0
```

```
$ java LoopPattern
```

```
Enter a length: 1
```

```
*
```

Exercise: Nested loops

```
$ java Square
```

```
Enter a size: 5
```

```
*****
```

```
*****
```

```
*****
```

```
*****
```

```
*****
```

```
$ java Square
```

```
Enter a size: 1
```

```
*
```

```
$ java Square
```

```
Enter a size: 0
```